In [1]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

Load Dataset

In [2]:
```
df=pd.read_excel(r'C:\Users\anupa\Desktop\Python Learning\Cluster Analysis\Online Retail.xlsx')
```

In [3]:
```
# Display first 5 rows
df.head()
```

Out[3]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom

Basic Dataset Information

In [4]:
```
# Check dataset shape
df.shape
```

Out[4]:
```
(541909, 8)
```

In [6]:
```
# Check column names and data types
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   InvoiceNo    541909 non-null  object
 1   StockCode    541909 non-null  object
 2   Description  540455 non-null  object
 3   Quantity     541909 non-null  int64
 4   InvoiceDate  541909 non-null  datetime64[ns]
 5   UnitPrice    541909 non-null  float64
 6   CustomerID   406829 non-null  float64
 7   Country      541909 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 33.1+ MB
```

In [5]:
```
# Statistical summary
df.describe()
```

Out[5]:

	Quantity	InvoiceDate	UnitPrice	CustomerID
count	541909.000000	541909	541909.000000	406829.000000
mean	9.552250	2011-07-04 13:34:57.156386048	4.611114	15287.690570
min	-80995.000000	2010-12-01 08:26:00	-11062.060000	12346.000000
25%	1.000000	2011-03-28 11:34:00	1.250000	13953.000000
50%	3.000000	2011-07-19 17:17:00	2.080000	15152.000000
75%	10.000000	2011-10-19 11:27:00	4.130000	16791.000000
max	80995.000000	2011-12-09 12:50:00	38970.000000	18287.000000
std	218.081158	NaN	96.759853	1713.600303

Handle Missing Values

In [7]:
```
# Check missing values
df.isnull().sum()
```

Out[7]:
```
InvoiceNo           0
StockCode           0
Description      1454
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     135080
Country             0
dtype: int64
```

In [8]:
```
# Remove rows with missing CustomerID
df = df.dropna(subset=['CustomerID'])
```

Remove Negative or Invalid Transactions

In [9]:
```
# Remove negative or zero quantities
df = df[df['Quantity'] > 0]
```

In [10]:
```
# Remove negative prices
df = df[df['UnitPrice'] > 0]
```

Convert Date Column

In [11]:
```
# Convert InvoiceDate to datetime format
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
```

Create Total Purchase Amount

In [12]:
```
# Create a new column for total price

df['TotalPrice'] = df['Quantity'] * df['UnitPrice']

df.head()
```

Out[12]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	TotalPrice
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom	15.30
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom	20.34
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom	22.00
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom	20.34
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom	20.34

Find Latest Transaction Date

In [13]:
```
# Determine the most recent transaction date
snapshot_date = df['InvoiceDate'].max()
snapshot_date
```

Out[13]:
```
Timestamp('2011-12-09 12:50:00')
```

Create RFM Table

RFM metrics:

Recency → Days since last purchase

Frequency → Number of purchases

Monetary → Total money spent

In [14]:
```
# Group data by CustomerID to calculate RFM metrics

rfm = df.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (snapshot_date - x.max()).days,
    'InvoiceNo': 'nunique',
    'TotalPrice': 'sum'
})

rfm
```

Out[14]:

CustomerID	InvoiceDate	InvoiceNo	TotalPrice
12346.0	325	1	77183.60
12347.0	1	7	4310.00
12348.0	74	4	1797.24
12349.0	18	1	1757.55
12350.0	309	1	334.40
...	...	...	...
18280.0	277	1	180.60
18281.0	180	1	80.82
18282.0	7	2	178.05
18283.0	3	16	2094.88
18287.0	42	3	1837.28

4338 rows × 3 columns

Rename Columns

In [15]:

```
# Rename columns for clarity

rfm.columns = ['Recency', 'Frequency', 'Monetary']

rfm.head()
```

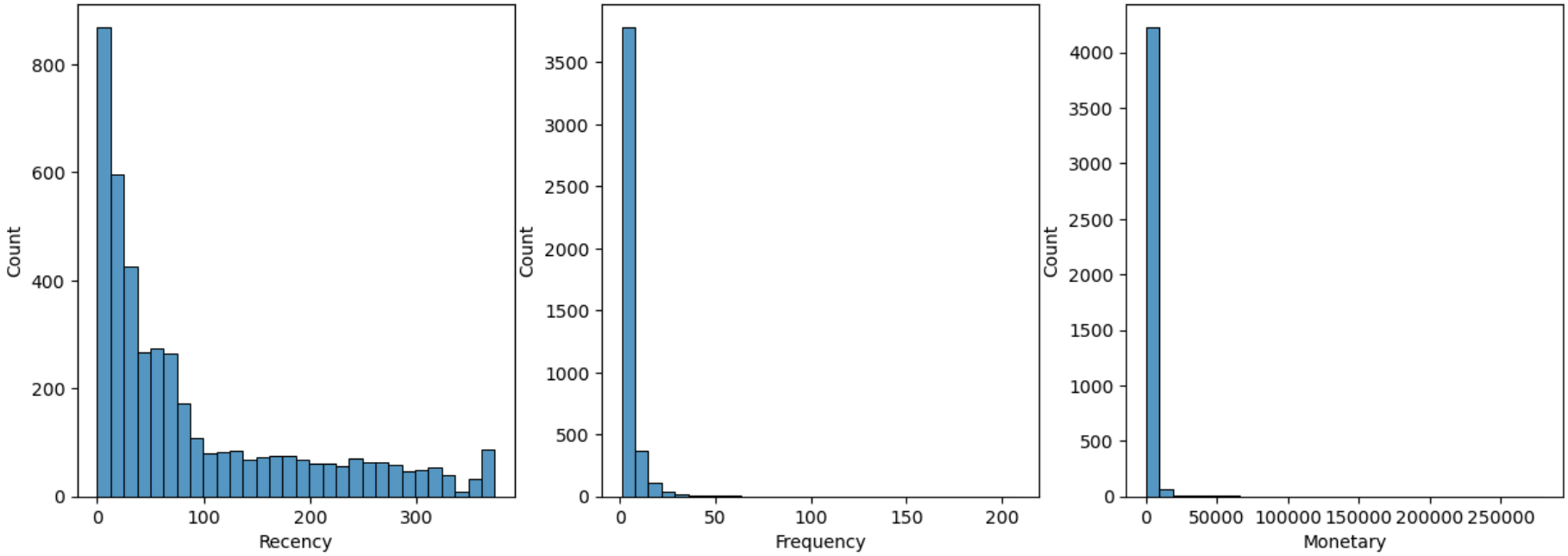
Out[15]:

CustomerID	Recency	Frequency	Monetary
12346.0	325	1	77183.60
12347.0	1	7	4310.00
12348.0	74	4	1797.24
12349.0	18	1	1757.55
12350.0	309	1	334.40

Check Distribution

In [17]:

```
# Plot distribution of RFM features

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
sns.histplot(rfm['Recency'], bins=30)

plt.subplot(1,3,2)
sns.histplot(rfm['Frequency'], bins=30)

plt.subplot(1,3,3)
sns.histplot(rfm['Monetary'], bins=30)

plt.show()
```



Feature Scaling

In [18]:
```
scaler = StandardScaler()

rfm_scaled = scaler.fit_transform(rfm)

rfm_scaled = pd.DataFrame(rfm_scaled, columns=rfm.columns)

rfm_scaled.head()
```

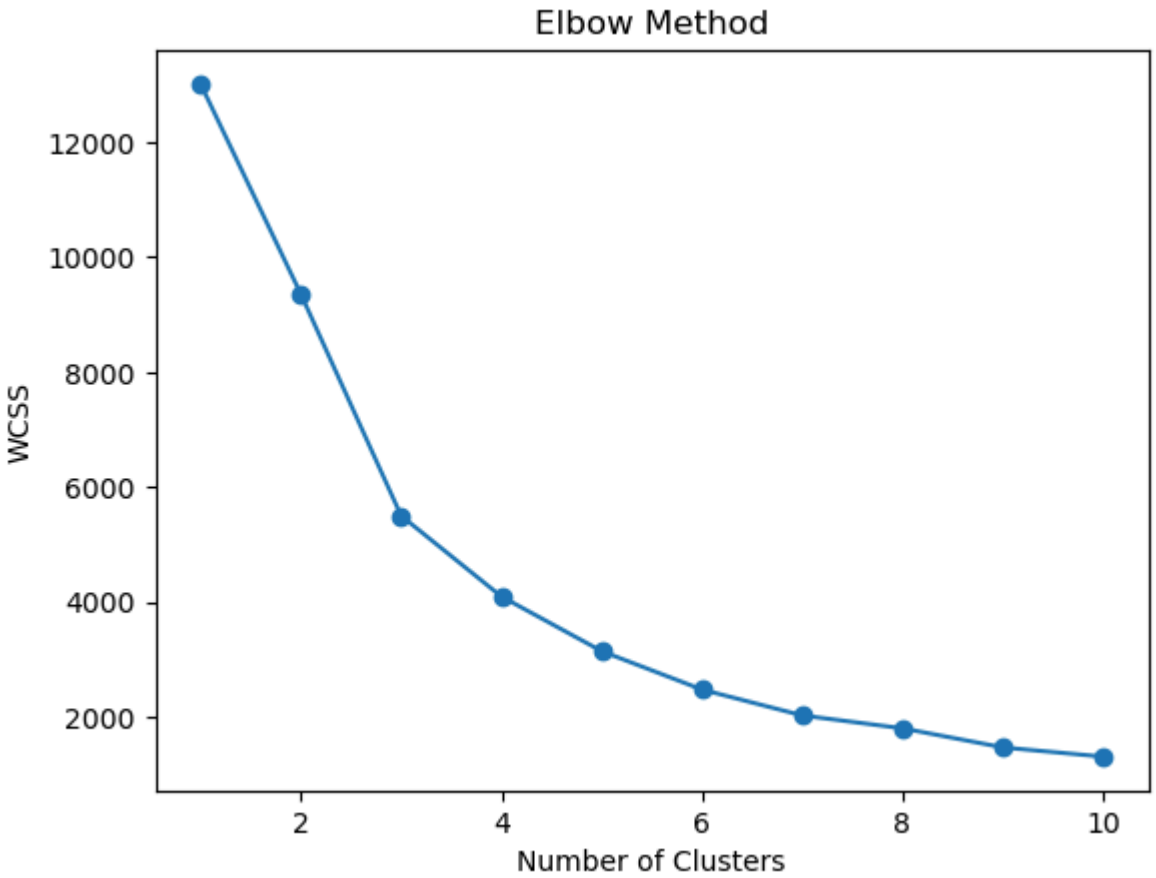
Out[18]:

	Recency	Frequency	Monetary
0	2.334574	-0.425097	8.358668
1	-0.905340	0.354417	0.250966
2	-0.175360	-0.035340	-0.028596
3	-0.735345	-0.425097	-0.033012
4	2.174578	-0.425097	-0.191347

Elbow Method (Find Optimal Clusters)

In [19]:

```
wcss = []

for i in range(1,11):

    kmeans = KMeans(n_clusters=i, random_state=42)

    kmeans.fit(rfm_scaled)

    wcss.append(kmeans.inertia_)

plt.plot(range(1,11), wcss, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```



Apply KMeans Clustering

In [30]:
```
kmeans = KMeans(n_clusters=4, random_state=42)

rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)

rfm.head()
```

Out[30]:

CustomerID	Recency	Frequency	Monetary	Cluster
12346.0	325	1	77183.60	3
12347.0	1	7	4310.00	0
12348.0	74	4	1797.24	0
12349.0	18	1	1757.55	0
12350.0	309	1	334.40	1

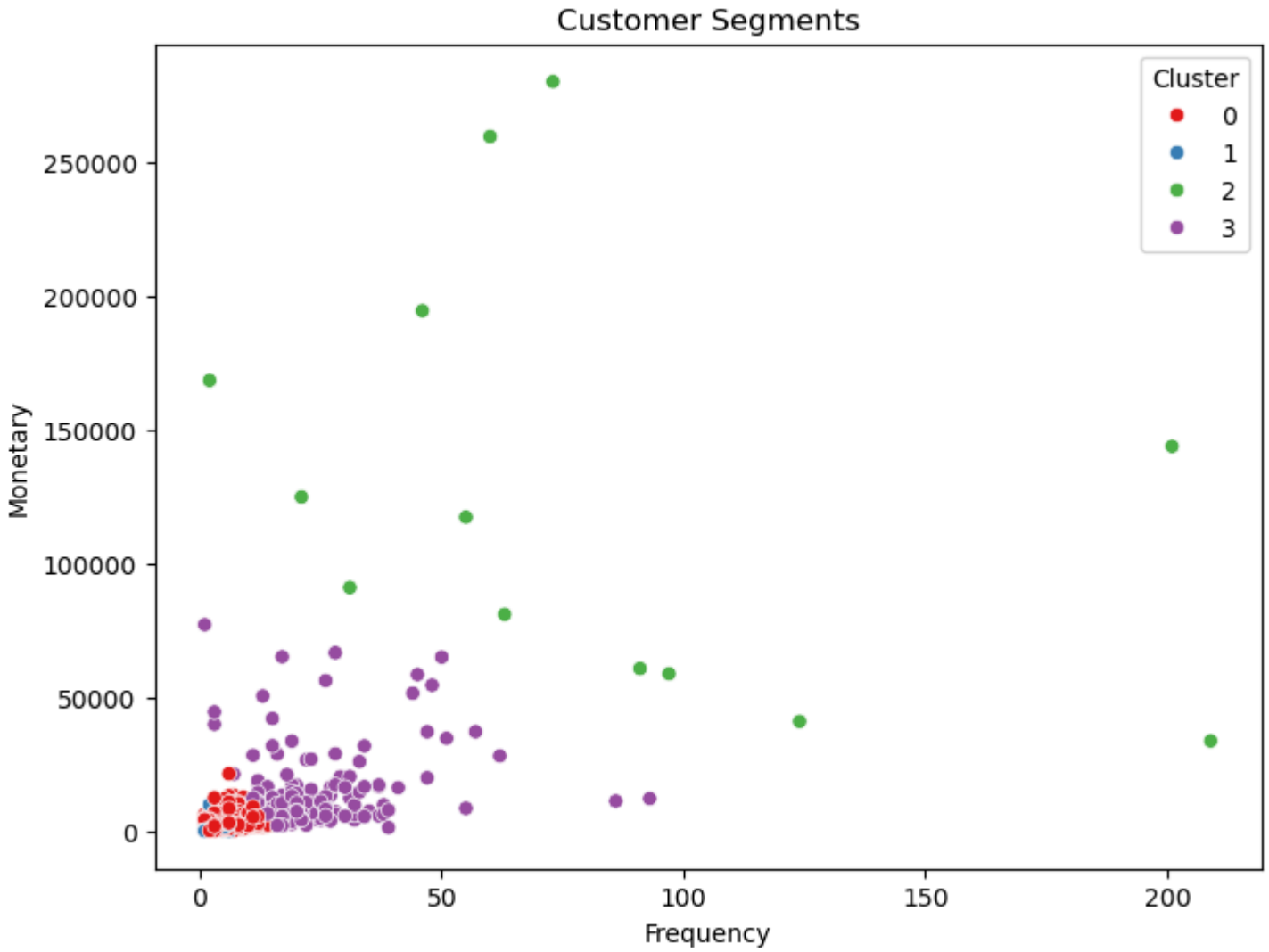
Evaluate Clustering

In [31]:
```
score = silhouette_score(rfm_scaled, rfm['Cluster'])

print("Silhouette Score:", score)
```

```
Silhouette Score: 0.616212846765192
```

Visualize Clusters

In [32]:
```
plt.figure(figsize=(8,6))

sns.scatterplot(
    x=rfm['Frequency'],
    y=rfm['Monetary'],
    hue=rfm['Cluster'],
    palette='Set1'
)

plt.title("Customer Segments")
plt.show()
```



Cluster Summary

In [33]:
```
cluster_summary = rfm.groupby('Cluster').mean()

cluster_summary
```

Out[33]:

Cluster	Recency	Frequency	Monetary
0	42.702685	3.682711	1359.049284
1	247.075914	1.552015	480.617480
2	6.384615	82.538462	127338.313846
3	14.500000	22.333333	12709.090490

Business Insights

Cluster 0 → Premium customers → loyalty programs

Cluster 1 → Regular buyers → targeted promotions

Cluster 2 → Lost customers → re-engagement campaigns

Cluster 3 → Price-sensitive buyers → discounts